HW 2: ROSAutonomous Mobile Robot Navigation

Christine Marie Lirazan
Department of Mechatronics Engineering, Kennesaw State University
clirazan@students.kennesaw.edu

***Abstract*—This homework focused on implementing SLAM and Path Planning using ROSBOT 2.0 in the Gazebo simulator. The robot was configured, used for mapping, and performed autonomous navigation. The system was verified using ROS tools including tf, rqt, ROS services, ROS actions, ROS bag, and ROS time.**

***Keywords*—ROS, SLAM, autonomous navigation, path planning, mobile robotics, robot localization, Gazebo simulation, RViz, ROS navigation stack, occupancy grid mapping.**

I. INTRODUCTION

The objective of this assignment was to implement SLAM and path planning using ROSBOT 2.0 in Gazebo. SLAM enables a robot to build a map of an unknown environment while estimating its position, while path planning enables autonomous navigation to target locations using that map. The system was implemented using multiple launch files and verified using standard ROS debugging tools.

II. METHODOLOGY

The assignment was completed using three main launch files: launch_go.launch, launch_go_2.launch, and path_go.launch.

A. launch_go.launch (Initial SLAM Mapping)

The launch_go.launch file was used to initialize the ROSBOT 2.0 environment in Gazebo and RVIZ and begin SLAM mapping. The robot explored the environment while generating an occupancy grid map using sensor data. A custom driver_controller.cpp file was used for robot motion during mapping.

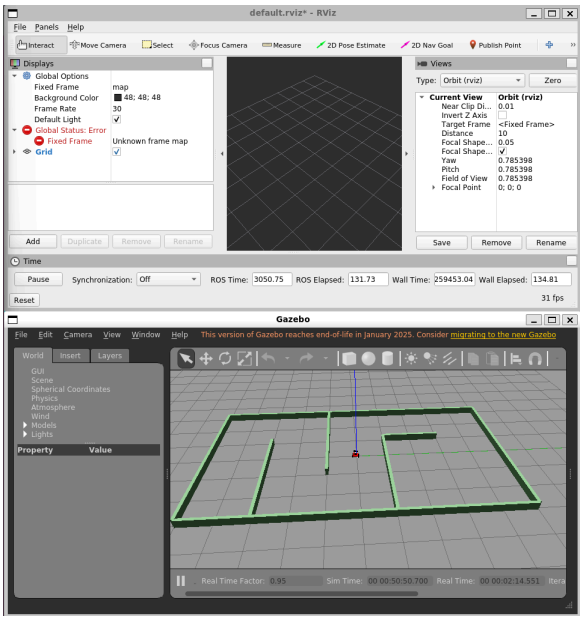

Fig. 1. Gazebo and RVIZ Initialization

B. launch_go_2.launch (SLAM Refinement)

The launch_go_2.launch file was used to continue SLAM mapping and improve the accuracy of the generated map through additional exploration of the environment.

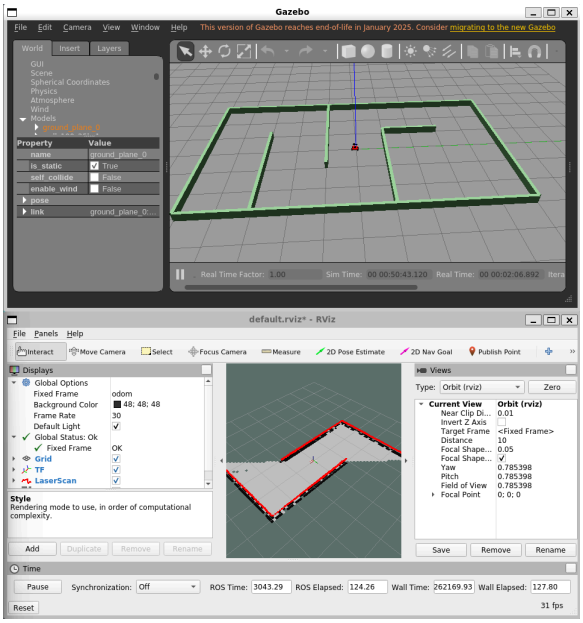

Fig. 2. SLAM Mapping in Progress

C. path_go.launch (Path Planning and Navigation)

The path_go.launch file was used for autonomous navigation after SLAM completion. A navigation stack was configured using costmap and trajectory planner YAML files, allowing the robot to generate and follow a collision-free path to a target goal in RVIZ.

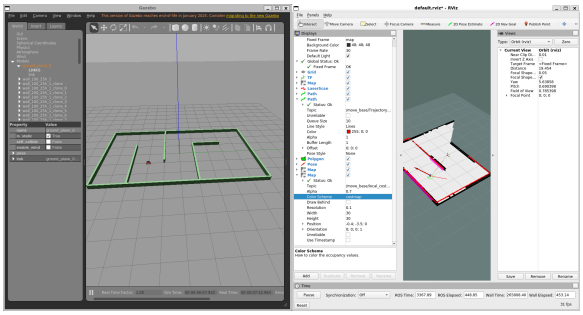

Fig. 3. Gazebo and RVIZ Initialization

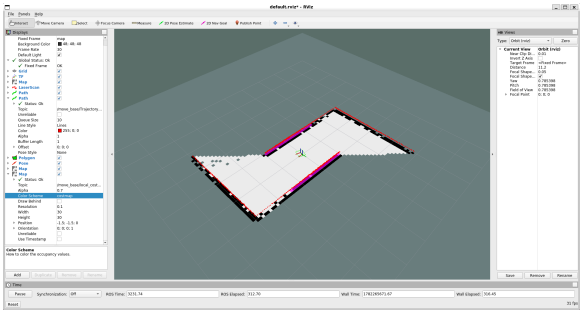

Fig. 4. RVIZ Navigation Settings and Path Planning

D. Verification and Testing

The system was verified using ROS debugging tools. The tf tool confirmed correct transformations between map, odom, and base_link frames. rqt_graph was used to verify node and topic communication.

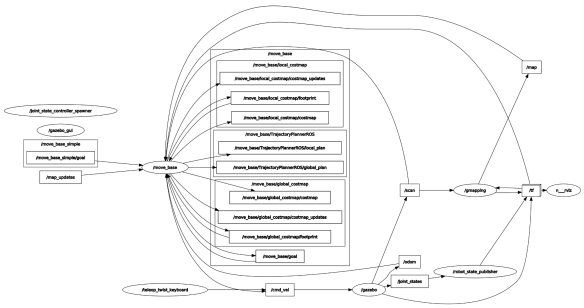

Fig. 5. Rqt_graph

E. Video Demonstration

A zip file containing recorded videos for all three launch files (launch_go.launch, launch_go_2.launch, and path_go.launch) is included in the submission.

F. Note

The Husarion ROSBOT tutorials referenced in the lecture videos have been updated and do not fully match the versions shown in class. Therefore, the training videos were used as the primary reference when completing SLAM and path planning tasks.

III. RESULTS

The ROSBOT 2.0 successfully generated a map using SLAM and performed autonomous navigation using path planning. The robot was able to reach target locations while avoiding obstacles. All ROS verification tools confirmed correct system operation and communication.

IV. CONCLUSION

This assignment demonstrated successful implementation of SLAM and path planning using ROSBOT 2.0 in Gazebo. The robot generated a map, localized itself, and executed autonomous navigation. The use of separate launch files provided a structured workflow for mapping and navigation.

V. REFERENCES

[1] M. H. Tanveer, "Training SLAM," course lecture video, Department of Robotics & Mechatronics Engineering, Kennesaw State University, accessed Jun. 16, 2026.

[2] Husarion, "ROS Tutorials (ROS1)," Available: https://husarion.com/tutorials/ros-tutorials/ros1/. Accessed: Jun. 23, 2026.

[3] ROS Wiki, "ROS Tutorials," ROS Documentation. Available: https://wiki.ros.org/ROS/Tutorials. Accessed: Jun. 23, 2026.